What Does a Streaming Client See When an LLM Server Crashes?

A Fault-Injection Study of Client-Visible Token Anomalies in vLLM and SGLang

Rajshekar Medipally
Independent researcher, Richmond, VA
https://github.com/rmedipallycic/llm-inference-fault-tolerance

Draft, October 2026

Abstract

LLM inference servers stream generated tokens to clients over requests that last seconds to minutes. When a serving process fails mid-generation, the client is left holding a partial response, and the recovery it performs determines what it ultimately shows to a user or passes to a downstream agent. Prior work on fault-tolerant LLM serving measures server-side recovery time. This paper instead measures what the client receives. We define three client-visible anomalies (duplicated, missing, and divergent text) relative to a failure-free reference, build an open-source fault-injection harness that SIGKILLs an unmodified OpenAI-compatible server at a chosen token position and records every streamed chunk, and apply it to vLLM and SGLang serving Llama 3.1 8B Instruct on a single GPU. We compare the two recovery strategies available to a client of an unmodified engine: naive retry, which resubmits the original prompt, and recompute, which resubmits the prompt plus the text already delivered.

PENDING MEASURED DATA. One to two sentences stating the measured anomaly rates and recovery latencies for each engine, strategy, and temperature.

All raw logs and the single analysis script are public.

1 Introduction

LLM serving has become stateful streaming infrastructure. A generation request holds a growing key-value (KV) cache on an accelerator and emits tokens to the client incrementally. Failure handling, however, is still largely stateless: if the serving process dies, the client or a load balancer resubmits the request and generation starts again.

That recovery has a cost that server-side metrics miss. A client that has already displayed 128 tokens may receive those 128 tokens a second time, may receive a continuation that disagrees with what it already showed, or may lose text at the boundary. For a chat interface this is a visible glitch. For an agent that has already acted on a partial tool call or a partial plan, it is a correctness problem. Stream processing systems name this property *delivery semantics* and engineer exactly-once output deliberately [5, 2, 1]. LLM serving has no equivalent definition, and to our knowledge no measurement of what clients actually receive.

This paper makes three contributions:

1. **Definitions.** Token-level delivery semantics for streamed generation, and three measurable client-visible anomalies: duplication, loss, and divergence (Section 3).
2. **A fault-injection harness** for any OpenAI-compatible server, which kills the full server process tree at a chosen chunk, restarts it, applies a client recovery strategy, and records every chunk with timestamps, the code commit, package versions, and GPU inventory (Section 4).
3. **A measurement study** of naive retry and recompute on vLLM and SGLang at temperature 0 and 0.8, at three kill positions (Sections 5–6).

The study deliberately uses unmodified engines and client-side recovery only. It establishes the baseline that a server-side exactly-once protocol, the next stage of this project, would have to improve on.

2 Background

Autoregressive serving. A request runs a prefill phase, which processes the prompt in parallel and builds the KV cache, followed by a decode phase that produces one token per step and streams it to the client. Engines such as vLLM [6] and SGLang [12] batch requests at iteration granularity [11] and manage the KV cache in fixed-size blocks.

The state lost on failure. The KV cache is a deterministic function of the weights and the token sequence, so it can always be rebuilt by prefill. What cannot be rebuilt from the prompt alone is the *sampling path*: at temperature above zero, the continuation depends on random draws whose state lives in the dead process.

Rollback-recovery vocabulary. Classic rollback-recovery distinguishes checkpointing from logging and defines *output commit*: a message to the outside world must not be released until the state that produced it is recoverable [5]. Mapped onto LLM serving, the token sequence is a replayable input log, the KV cache is derived operator state, and the streaming client is the external sink where delivery semantics are decided.

3 Delivery model and anomalies

Let G be the failure-free reference output for a request, recorded on the same server configuration. A failure occurs after the client has received delivered text D. After recovery the client receives recovery text R and its final view is $V = D \cdot R$ (concatenation), which is what a typical client displays.

We measure:

- **Duplication**: the length of the longest common prefix of R and D, reported as the fraction $dup = |\mathrm{lcp}(R, D)|/|D|$. $dup = 1$ means the client received all of D a second time.
- **Divergence**: whether the client's text disagrees with G after the duplicated prefix is removed, i.e. whether $V' = D \cdot R[dup{:}]$ and G differ at some position before either ends. Removing the duplicate first keeps the two anomalies separate: a retry that re-sends identical text is duplication, not divergence. The first differing index is recorded and compared with $|D|$, the kill boundary.
- **Exact match**: $V = G$, i.e. the failure was invisible in the final text.
- **Recovery latency**: wall time from the kill to the first chunk delivered after recovery, including server restart.

Under this model, *at-least-once* delivery shows as $dup > 0$, *at-most-once* as V being a strict prefix of G, and *exactly-once* as $V = G$. Comparison is on text, not token ids, because the OpenAI-compatible streaming API does not guarantee one token per chunk.

A comparison against G is only meaningful if the server is reproducible without failures. The harness therefore records two failure-free references per run and flags the run when they differ (Section 4).

Client recovery strategies. **Naive retry** resubmits the original prompt with the original `max_tokens`; many client libraries' retry wrappers do this. **Recompute** resubmits the prompt concatenated with D and requests the remaining token budget, so the server rebuilds the KV cache by prefill and continues. Neither requires engine changes.

4 Fault-injection harness

The harness (`harness/`, about 340 lines of Python) has three parts.

Server lifecycle. The server command is started in its own process group and declared ready when `GET /v1/models` returns 200. A fault is a `SIGKILL` of the whole process group, since vLLM and SGLang spawn worker processes that otherwise keep holding GPU memory and the port. On Windows, used only for laptop smoke tests, the equivalent is `taskkill /F /T`.

Streaming client. Requests go to `/v1/completions` with `stream=true`. Every server-sent event with non-empty text is recorded as a chunk with wall-clock and monotonic timestamps. The kill fires from the chunk callback when the configured chunk count is reached, and the client keeps reading, so chunks the server had already sent before dying are recorded as in-flight rather than lost.

Trial protocol. Each run (one engine, strategy, temperature, and kill position) starts the server, records two failure-free references G and G', then for each trial: streams the request, kills at the kill point, restarts the server and waits for readiness, issues the recovery request, and writes a JSON record containing every chunk, the kill time, the restart time, and the client view V.

Provenance. Each run records the git commit and whether the working tree was dirty, the full command line, installed versions of `vllm`, `sglang`, `torch`, and `transformers`, the GPU model, memory, and driver from `nvidia-smi`, and the full server log. Measured runs are written to `experiments/` and committed; harness smoke tests are written to a separate, git-ignored directory and are never reported. All statistics in this paper are produced by one script, `analysis/analyze.py`, from those committed files.

Validation. The harness was exercised end to end on a laptop CPU with llama.cpp and a 0.5B model before any GPU run. That check, documented in the repository as a mechanics check rather than a result, led to two changes: a duplication metric based on shared prefix fraction rather than any shared character, and the reference-reproducibility check, after two clean seeded runs on that server disagreed at temperature 0.8.

5 Methodology

Setup. One cloud machine with a single NVIDIA GPU of at least 24 GB, Llama 3.1 8B Instruct in bf16,

vLLM and SGLang each in a separate Python environment with default engine flags except a 4,096-token context cap (Llama 3.1's 128k default does not fit a 24 GB KV cache; every request here is far shorter). Because the machine image ships no CUDA toolkit, FlashInfer kernels that compile at startup were avoided: vLLM used PyTorch sampling (`VLLM_USE_FLASHINFER_SAMPLER=0`) and SGLang used Triton attention with PyTorch sampling. Both settings are recorded in every run log. The GPU model, driver, and package versions are recorded by the harness in every run and reported as recorded.

PENDING MEASURED DATA. Engine, torch, CUDA and driver versions and model revision hash, generated from `run.json` by `analysis/make_tables.py`.

Grid. For each engine: strategy $\in$ {naive retry, recompute} $\times$ temperature $\in \{0, 0.8\}$ $\times$ kill point $\in$ {25, 128, 230} chunks (about 10%, 50%, and 90% of a 256-token budget), with 10 trials per cell and a fixed per-request seed (1234). That is 12 runs and 120 trials per engine, 240 in total, produced by `scripts/run_matrix.sh`. Requests are issued one at a time, so there is no batching interference.

Hypotheses. Stated in advance so that the data can refute them:

H1 At temperature 0, naive retry yields $dup = 1$ in every trial and recompute yields $V = G$ in every trial.

H2 At temperature 0.8 with a fixed seed, both engines reproduce G across clean runs; if so, naive retry still yields full duplication and recompute yields divergence from G after the kill point, because the sampler restarts from a different random state.

H3 Recovery latency is dominated by server restart, not by prefill of the delivered text.

6 Results

PENDING MEASURED DATA. Reproducibility first. For each engine and temperature, report whether the two failure-free references matched (`reference repeat identical`). Any cell where they did not is marked uninterpretable for divergence and excluded from Table 1, with the count stated.

PENDING MEASURED DATA. H1, H2 outcomes by kill position, with the raw counts. State plainly where a hypothesis was refuted.

PENDING MEASURED DATA. H3: restart time versus recovery latency, and whether recompute is measurably slower than naive retry to first token at the 90% kill point.

Table 1: Client-visible anomalies per cell (10 trials each), generated from the raw logs by `analysis/make_tables.py`. Diverged′ is counted after removing duplicated text (Section 3).

Engine	Strategy	T	$V{=}G$	Full dup	Diverged′
vLLM	naive	0	–	–	–
vLLM	naive	0.8	–	–	–
vLLM	recomp.	0	–	–	–
vLLM	recomp.	0.8	–	–	–
SGLang	naive	0	–	–	–
SGLang	naive	0.8	–	–	–
SGLang	recomp.	0	–	–	–
SGLang	recomp.	0.8	–	–	–

Table 2: Restart and recovery latency, median (min–max), seconds.

Engine	Server restart	Kill → first recovered token
vLLM	–	–
SGLang	–	–

7 Discussion

PENDING MEASURED DATA. What the measured anomalies mean for chat clients and for agents that act on partial output; which strategy a client should use for each temperature; whether the difference between engines matters.

Why client-side recovery is not enough. Even in the best case for recompute, the client cannot restore the sampler's random state, so above temperature 0 it can at most hide duplication, not guarantee the continuation the original request would have produced. Exactly-once delivery therefore requires state the client does not hold: a sequence-numbered token log and the sampler state, committed before tokens are released. That protocol is the next stage of this project.

8 Related work

Fault-tolerant LLM serving. DéjàVu [8] streams and replicates KV cache to recover from failures and reports server-side recovery time; it does not characterize what streaming clients receive. SpotServe [7] migrates in-flight context when preemptible instances are reclaimed with warning, and Llumnix [9] live-migrates requests for load balancing. Both handle planned moves; this study targets unplanned fail-stop crashes. **Serving engines.** Orca [11] introduced iteration-level scheduling, vLLM [6] block-managed KV memory, and SGLang [12] prefix sharing via RadixAttention. **Delivery semantics.** Chandy and Lamport [3] define consistent snapshots, Elnozahy et al. [5] output commit, and Flink

[2] and Spark Structured Streaming [1] build exactly-once sinks on them. Optimal checkpoint intervals [10, 4] motivate later stages of this project.

9 Threats to validity

- **One model, one GPU, one prompt.** Results are scoped to Llama 3.1 8B on one single-GPU machine with a single explanatory prompt. Tensor-parallel serving and other prompts may differ.
- **Re-tokenization in recompute.** Recompute sends D as text, and the engine re-tokenizes it. If the boundary tokenizes differently than the original output tokens, the continuation can diverge at temperature 0 for a reason unrelated to sampling. Any such divergence is reported with its position relative to the kill point.
- **Fail-stop only.** `SIGKILL` is a clean crash; GPU hangs, NCCL stalls, and silent corruption are not modeled.
- **Latency includes restart.** With a single instance, recovery latency includes model reload; failover to a warm standby would be faster.
- **Chunks are approximate tokens.** Kill points are set in chunks, which are usually but not always one token.

10 Conclusion and future work

PENDING MEASURED DATA. Two-sentence summary of measured findings.

Next stages: (1) a gateway that assigns sequence numbers, logs tokens and sampler state before release, and deduplicates on recovery, evaluated on the same harness for zero anomalies and per-token overhead; (2) measured recompute-versus-checkpoint cost curves for the KV cache; (3) an adaptive per-request checkpoint policy.

Artifact. Code, raw logs, and the analysis script are at `https://github.com/rmedipallycic/llm-inference-fault-tolerance` (Apache-2.0). Every number above traces to a committed log file.

References

[1] Michael Armbrust, Tathagata Das, Joseph Torres, Burak Yavuz, Shixiong Zhu, Reynold Xin, Ali Ghodsi, Ion Stoica, and Matei Zaharia. Structured streaming: A declarative API for real-time applications in Apache Spark. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, pages 601–613, 2018.

[2] Paris Carbone, Gyula Fóra, Stephan Ewen, Seif Haridi, and Kostas Tzoumas. Lightweight asynchronous snapshots for distributed dataflows, 2015. arXiv:1506.08603.

[3] K. Mani Chandy and Leslie Lamport. Distributed snapshots: Determining global states of distributed systems. *ACM Transactions on Computer Systems*, 3(1):63–75, 1985.

[4] J. T. Daly. A higher order estimate of the optimum checkpoint interval for restart dumps. *Future Generation Computer Systems*, 22(3):303–312, 2006.

[5] E. N. Elnozahy, Lorenzo Alvisi, Yi-Min Wang, and David B. Johnson. A survey of rollback-recovery protocols in message-passing systems. *ACM Computing Surveys*, 34(3):375–408, 2002.

[6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

[7] Xupeng Miao, Chunan Shi, Jiangfei Duan, Xiaoli Xi, Dahua Lin, Bin Cui, and Zhihao Jia. SpotServe: Serving generative large language models on preemptible instances. In *Proceedings of ASPLOS*, pages 1112–1127, 2024.

[8] Foteini Strati, Sara McAllister, Amar Phanishayee, Jakub Tarnawski, and Ana Klimovic. DéjàVu: KV-cache streaming for fast, fault-tolerant generative LLM serving. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, pages 46745–46771, 2024.

[9] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic scheduling for large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

[10] John W. Young. A first order approximation to the optimum checkpoint interval. *Communications of the ACM*, 17(9):530–531, 1974.

[11] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 521–538, 2022.

[12] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2312.07104.